计算机的基本工作原理

核心内容

- 计算机是 0-1 世界
  - 解读 字节流 的核心原则 —— 上下文 + 编码规则
- 计算机基本工作原理
  - 存储程序 —— 自动执行
  - 计算机基本结构 —— 冯.诺依曼结构
  - 程序和指令的执行过程 —— 冯.诺依曼结构

课堂目标

- 对一个给定的字节流，能够按照要求，解读成某种类型的数据
- 能够给出机器指令和汇编语句的异同点
- 能够解释什么是汇编，什么是反汇编
- 能够描述计算机工作的基本原理
- 能够描述程序计数器的作用，程序计数器的更新方法，更新时机
- 能够描述程序的运行基本过程
- 掌握机器语言级程序调试的基本方法，学会使用反汇编窗口、寄存器窗口、内存窗口、变量监视窗口等

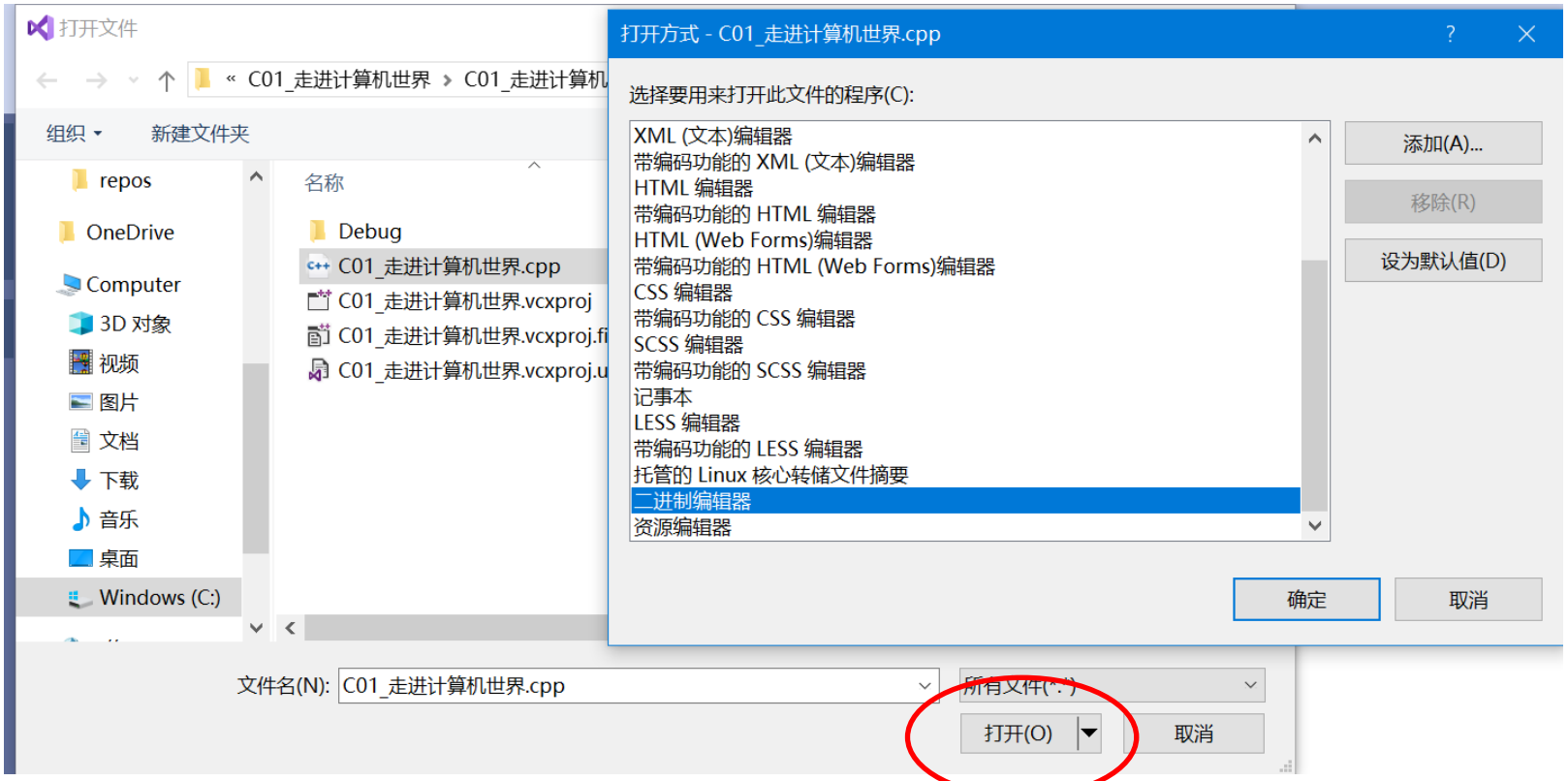
走进计算机的0、1世界

```
#include <stdio.h>

int main(int argc, char* argv[])
{
        int  x, y, z;
        x = 10;
        y = 20;
        z = 3 * x + 6 * y + 4 * 8;
        printf("3*%d+6*%d+4*8=%d\n", x, y, z);
        return 0;
}
```

走进计算机的0、1世界

◆ 以二进制形式观察源程序、执行程序



走进计算机的0、1世界

◆ 以二进制形式观察源程序

C01_走进计算机世界.cpp

```
00000000  23 69 6E 63 6C 75 64 65  20 3C 73 74 64 69 6F 2E  #include <stdio.
00000010  68 3E 0D 0A 0D 0A 69 6E  74 20 6D 61 69 6E 28 69  h>....int main(i
00000020  6E 74 20 61 72 67 63 2C  20 63 68 61 72 2A 20 61  nt argc, char* a
00000030  72 67 76 5B 5D 29 0D 0A  7B 0D 0A 09 69 6E 74 20  rgv[])..{...int
00000040  20 78 2C 20 79 2C 20 7A  3B 0D 0A 09 78 20 3D 20   x, y, z;...x =
00000050  31 30 3B 0D 0A 09 79 20  3D 20 32 30 3B 0D 0A 09  10;...y = 20;...
00000060  7A 20 3D 20 33 20 2A 20  78 20 2B 20 36 20 2A 20  z = 3 * x + 6 *
00000070  79 20 2B 20 34 20 2A 20  38 3B 0D 0A 09 70 72 69  y + 4 * 8;...pri
00000080  6E 74 66 28 22 33 2A 25  64 2B 36 2A 25 64 2B 34  ntf("3*%d+6*%d+4
00000090  2A 38 3D 25 64 5C 6E 22  2C 20 78 2C 20 79 2C 20  *8=%d\n", x, y,
000000a0  7A 29 3B 0D 0A 09 72 65  74 75 72 6E 20 30 3B 0D  z);...return 0;.
000000b0  0A 7D 0D 0A                                       .}..
```

ASCII 码 ： American Standard Code for Information Interchange

0D 回车　0A 换行　20 空格

61 字符 a　按字母顺序编码 62 63 64 ……　十六进制

30 字符 0　按数字顺序编码 31 32 33 ……

Q： 最左边一列表示什么意思？

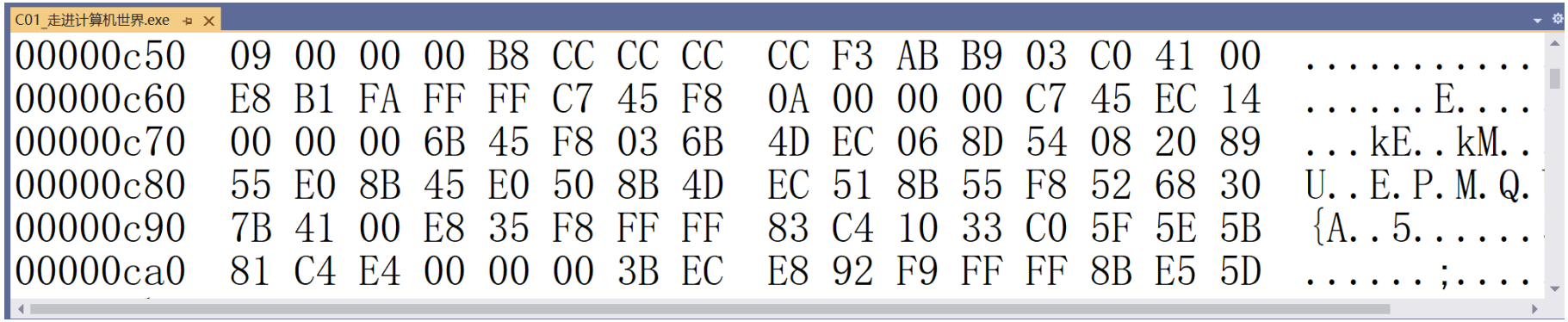
最右边的一列表示什么意思？

对应行的首字节在文件中的位置 （地址）

以字符形式显示字节内容

走进计算机的0、1世界

◆ 以二进制形式观察执行程序



与观察源程序的窗口一样，也是三个部分。

左：地址

中：字节流

右：以字符形式显示字节内容

执行程序中包含：机器指令、全局变量初始值、只读数据 ……(可执行目标文件格式)

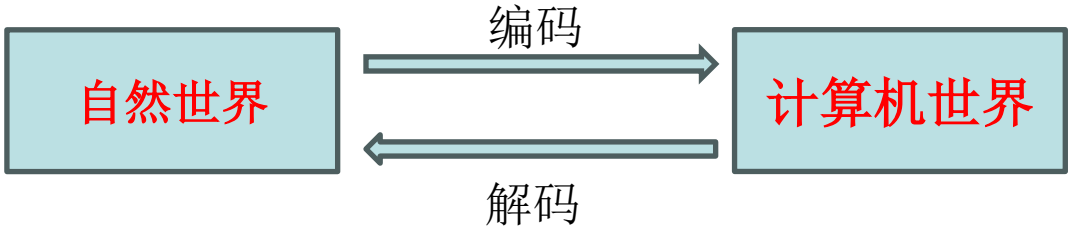
走进计算机的0、1世界

以二进制形式打开一个图像、word文件、PPT文件、声音文件、视频文件、网页.....

会看到什么？

走进计算机的0、1世界

计算机是0-1的世界。8个二进制位形成一个字节。一个个字节顺序排列在一起，形成字节流。



讨论：不同的内容有相同的编码，

如何能从一个字节流解码出不同的内容？

一个字节流又如何分成一系列小段？

走进计算机的0、1世界

身高一米七五年收入二十万人老实话不多

身高一米七五，年收入二十万，人老实，话不多。

身高一米七，五年收入二十万，人老，实话不多。

走进计算机的0、1世界

十六进制的字节流

31 32 33 34 35 36 37 38 39 30 31 00

```
string 12345678901
int 0x34333231  0x38373635  0x313039
int 875770417  943142453  3223609
short 0x3231  0x3433  0x3635 ...
short 12849  13363  13877 ...
float 0.000000  0.000044  0.000000
double 0.000000
```

当成字符串： 遇到 00，串结束

当成 int 数组：每个 int 元素占 4个字节

当成 short 数组：每个 short元素占 2个字节

走进计算机的0、1世界

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>     #include <string.h>
union share_space {
    char   string[12];
    int      x[3];
    short  y[6];
    float    f[3];
    double d;
};
int main()
{   share_space  test;
    strcpy(test.string, "12345678901");
    printf("string %s\n", test.string);
    printf("int %#x  %#x  %#x \n", test.x[0],test.x[1],test.x[2]);
    printf("int %d  %d  %d \n", test.x[0], test.x[1], test.x[2]);
    printf("short %#x  %#x  %#x ...\n", test.y[0], test.y[1], test.y[2]);
    printf("short %d  %d  %d ...\n", test.y[0], test.y[1], test.y[2]);
    printf("float %f  %f  %f \n", test.f[0], test.f[1], test.f[2]);
    printf("double %lf \n", test.d);
    return 0;
}
```

走进计算机的0、1世界

总结：

- 计算机是0-1的世界
- 编码是有一定规则的
- 解码时依赖上下文环境，即指定解释成什么信息
- 上下文 = 约定好的 “解读规则”（运行环境）

华为 2025年公开三进制逻辑门专利，核心是动态阈值三电平电路，适配现代 CMOS 工艺，用 27 种单变量函数简化电路，在模糊决策场景实测功耗降 42%。

量子计算机？

走进计算机的0、1世界



长线、短线，两种基元的组合

击鼓传令：殷墟出土的甲骨文记载了最早的组织通信活动就是通过击鼓，用声音来传报边境的军情。守鼓兵丁立即敲击大鼓，通过鼓点的间隔节奏来表示不同的内容。

烽火通信：烽火作为一种原始的声光通信手段，服务于古代军事战争。当敌人入侵时，便一个接一个地点燃起烽火报警，各路诸侯见到烽火，马上派兵相助，抵抗敌人。

走进计算机的0、1世界

现代二进制体系公认由莱布尼茨正式建立

1703年， 莱布尼茨发明二进制记数系统

发表论文《论只使用符号 0 和 1 的二进制算术》

- 只用 0、1 两个数字，就可以表示所有整数，并且建立了完整的加减乘除运算法则。

- 当时只是一项数学、哲学层面的研究。

走进计算机的0、1世界

布尔代数：二进制和逻辑挂上钩（1847-1854）

乔治·布尔 创立布尔代数。

不再只用 0、1 代表数字；

0 = 假(False)　1 = 真(True)

运算：与、或、非

布尔代数后来成为数字电路最核心的理论，
没有布尔代数，二进制就无法用于计算机。

走进计算机的0、1世界

香农打通理论-电路（1938）

香农：硕士论文《继电器与开关电路的符号分析》

开关电路的通、断两种状态，对应布尔代数的 1、0。

二进制从纯数学变成硬件可以实现的方案。

走进计算机的0、1世界

- 早期机电计算机直接用 0-9 十个状态。

- ENIAC（1945）：世界第一台电子计算机，十进制，不是二进制。

  - 并非用 10 档不同电压 表示 0-9。
  - 区分电压等级抗干扰极差，不可行。
  - 用 10 个二态元件，用 “哪个元件被激活” 代表数字（独热编码 one-hot）
  - 某一个元件打开，其余关闭，以此代表 0-9 当中的一个数字。

走进计算机的0、1世界

早期十进制计算机，0-9 十个状态的 4 种实现方案

机器类型	存储 0-9 原理	核心表示方法
巴贝奇差分机、机械计算器	十进制齿轮位置	物理位置
Harvard Mark-I（继电器机电）	10 个继电器，一个吸合	独热编码（继电器）
ENIAC（真空管）	10 根真空管环形计数器，一根导通	独热编码（电子管）
后期十进制设备 (Dekatron)	十进计数管点亮对应数字电极	专用十进制电子器件

走进计算机的0、1世界

冯·诺依曼在 EDVAC 报告（1945）明确提出：

计算机内部数据**采用二进制存储**。

- 电子元件最容易实现两种稳定状态：通电 / 断电，高电平 / 低电平。
- 电路简单，成本低、抗干扰强。
- 第一台真正实现**存储程序 + 二进制**的计算机：**曼彻斯特 Baby（1948）**，随后 EDVAC 完成。
- 晶体管：开关状态天然二值，完美适配二进制
- 集成电路、CPU、内存，每一个存储单元本质保存 0/1

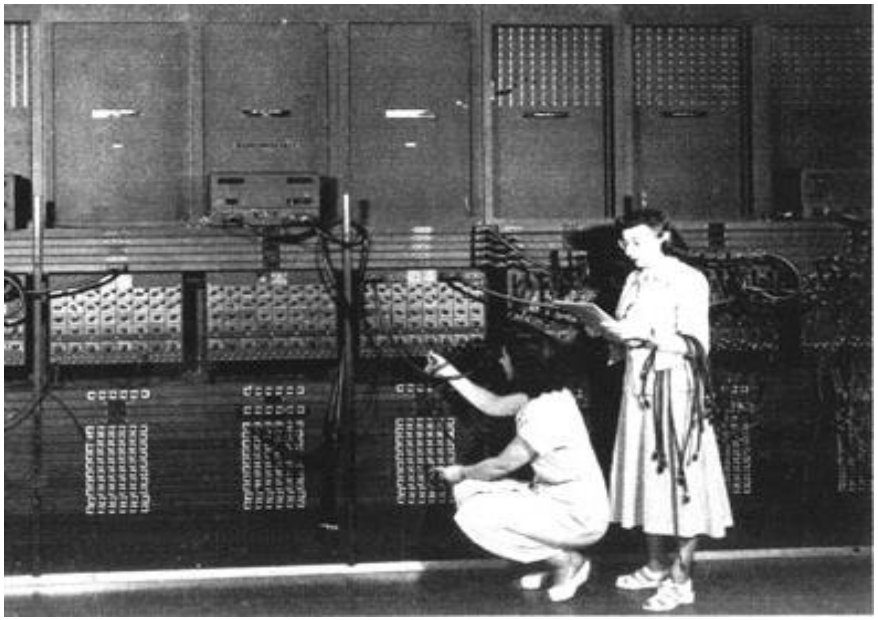
走进计算机的0、1世界——小结

年份	事件
1703	莱布尼茨正式提出二进制算术
1854	布尔创立布尔代数（0/1 代表真假）
1938	香农证明开关电路可以实现布尔逻辑
1945	ENIAC 诞生（十进制）；冯·诺依曼提出计算机改用二进制
1948-1951	第一批二进制存储程序计算机问世，二进制成为工业标准
1950~ 现在	晶体管、集成电路，二进制统治所有通用计算机

计算机工作基本原理

存储程序——自动执行

- 任何要计算机完成的工作都要先被编写成**程序**；
- 然后将程序和原始数据送入主存并启动执行。
- 一旦程序被启动，计算机应能在不需操作人员干预下，**自动**完成逐条取出指令和执行指令的任务。



与ENIAC的差别

Electronic Numerical Integrator And Computer

电子数字积分计算机

采用手动编程，通过设置开关和插拔电缆来实现

计算机工作基本原理



人工插拔接线　VS　程控交换机

计算机工作基本原理

可执行的程序：由一条条机器指令组成。

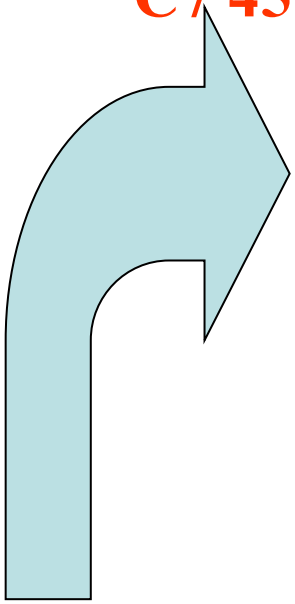
机器指令 (Instruction)，也称硬指令

- 由0和1组成的二进制代码、 能为计算机识别并执行，指挥计算机完成某一基本操作的命令。
- 依赖于某一类型的机器，取决于CPU。

计算机工作基本原理

例：将 某单元（[ebp-8]） 中的内容 置为 10。

C7 45 F8 0A 00 00 00



1100 0111
0100 0101
1111 1000
0000 1010
0000 0000
……

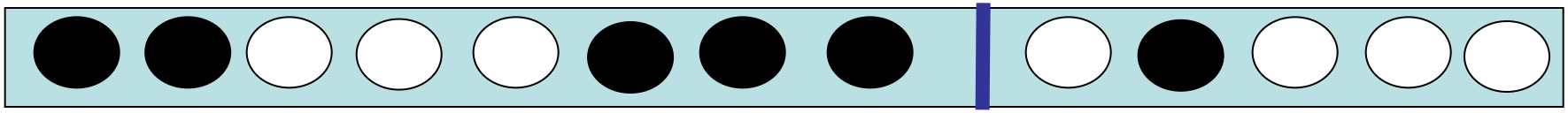
猜猜看：机器指令中的数值各代表什么含义？

0A 00 00 00 -> 10

F8 -> -8，局部变量x在栈中的位置；

45 -> 寻址方式

C7 -> 操作码

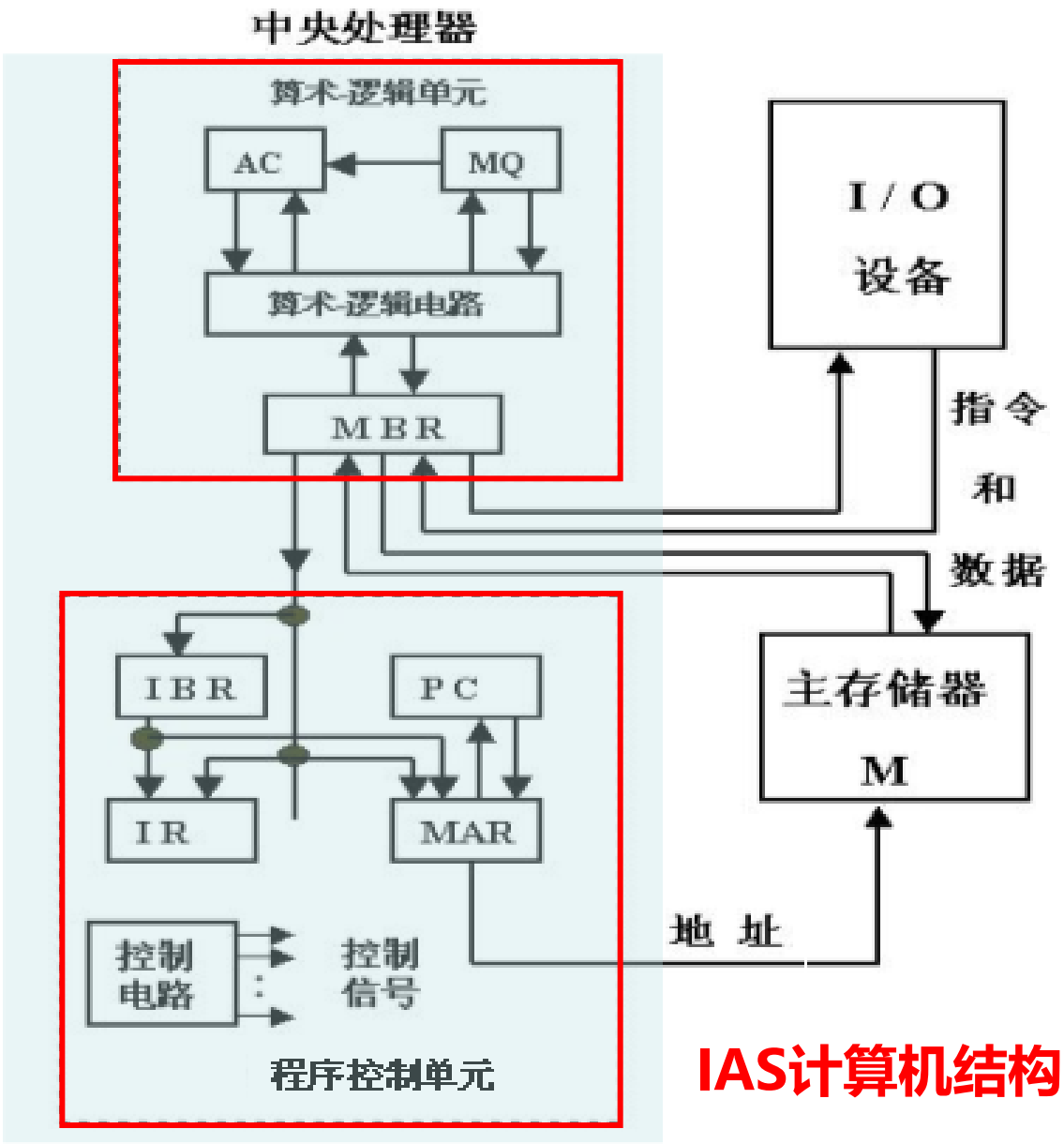


冯·诺依曼结构

- 应该有个主存，用来存放程序和数据
- 应该有一个自动逐条取出指令的部件
- 还应该有具体执行指令（即运算）的部件
- 程序由指令构成
- 指令描述如何对数据进行处理
- 应该有将程序和原始数据输入计算机的部件
- 应该有将运算结果输出计算机的部件

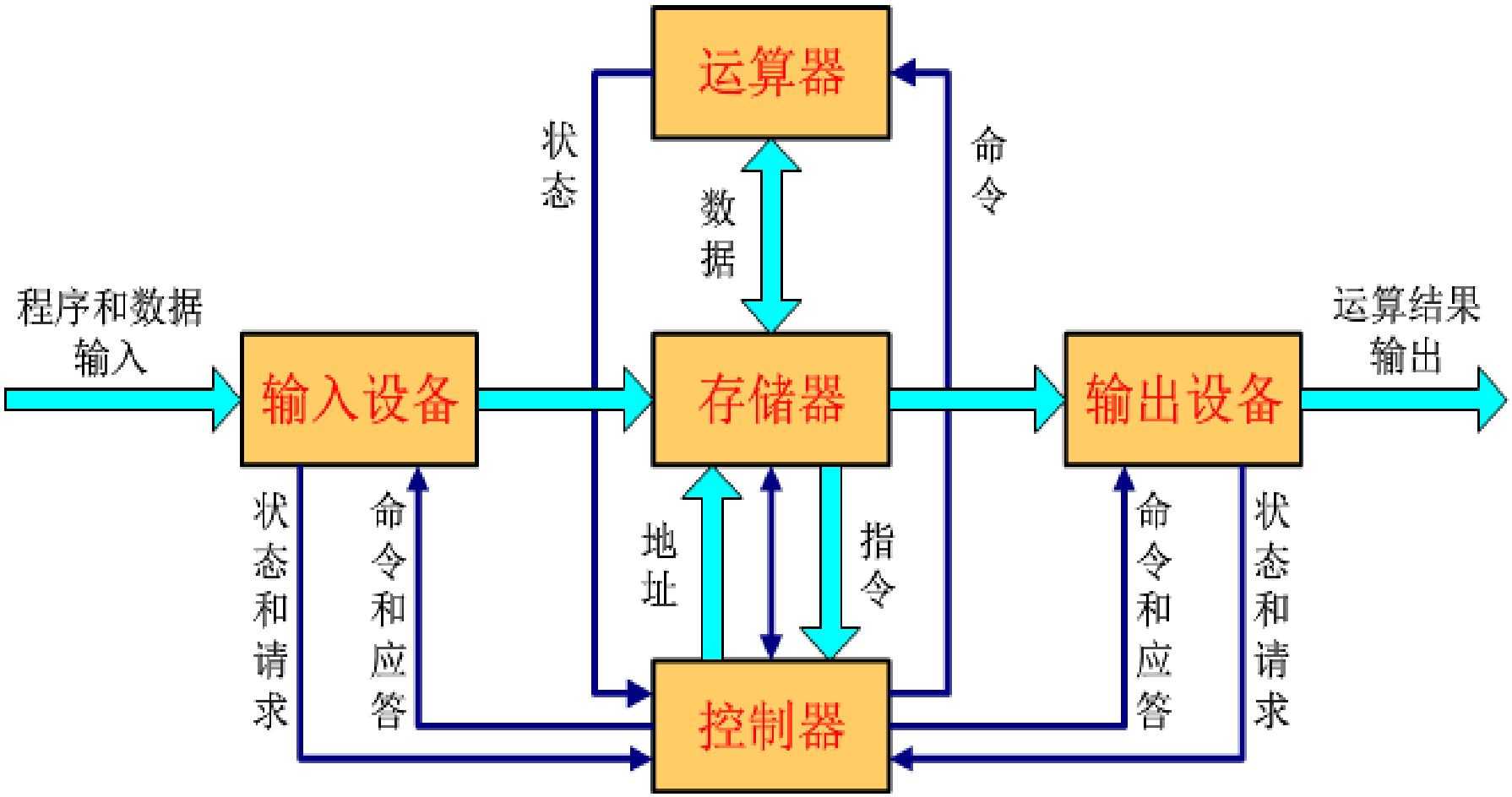
你还能想出更多吗?

你猜得八九不离十了☺



IAS计算机结构

冯.诺依曼结构计算机模型



早期：部件之间用分散方式相连

现在：部件之间大多用总线方式相连

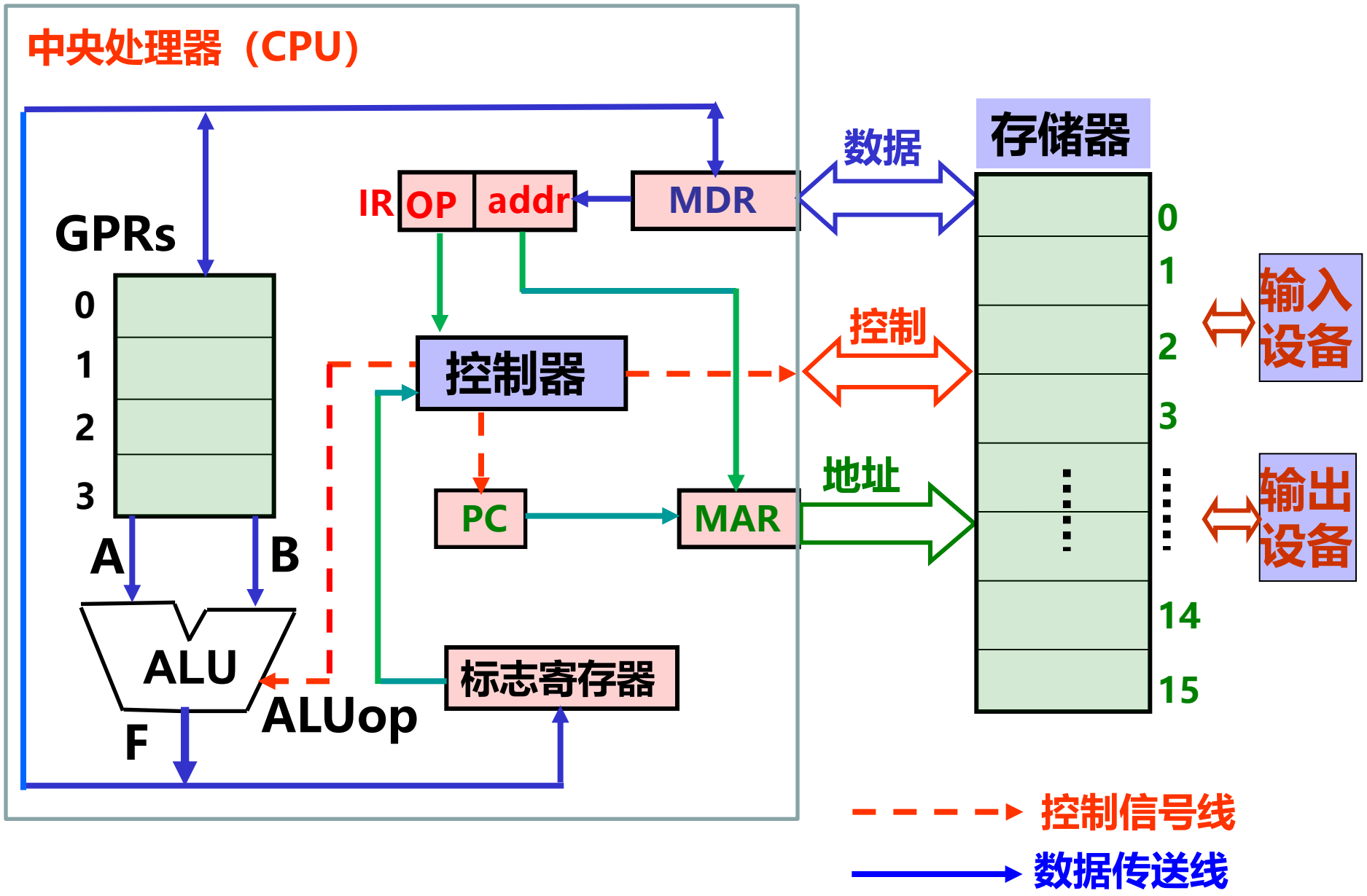
趋势：点对点（分散方式）高速连接

冯·诺依曼结构的主要思想

冯•诺依曼结构的主要思想是什么呢?

1. 计算机应由运算器、控制器、存储器、输入设备和输出设备五个基本部件组成。
2. 各基本部件的功能是:
   - 存储器不仅能存放数据，而且也能存放指令，形式上两者没有区别，但计算机应能区分数据还是指令;
   - 控制器应能自动取出指令来执行;
   - 运算器应能进行加/减/乘/除四种基本算术运算，并且也能进行一些逻辑运算和附加运算;
   - 操作人员可以通过输入设备、输出设备和主机进行通信。
3. 内部以二进制表示指令和数据。每条指令由操作码和地址码两部分组成。操作码指出操作类型，地址码指出操作数的地址。由一串指令组成程序。
4. 采用“存储程序”工作方式。

现代计算机结构模型

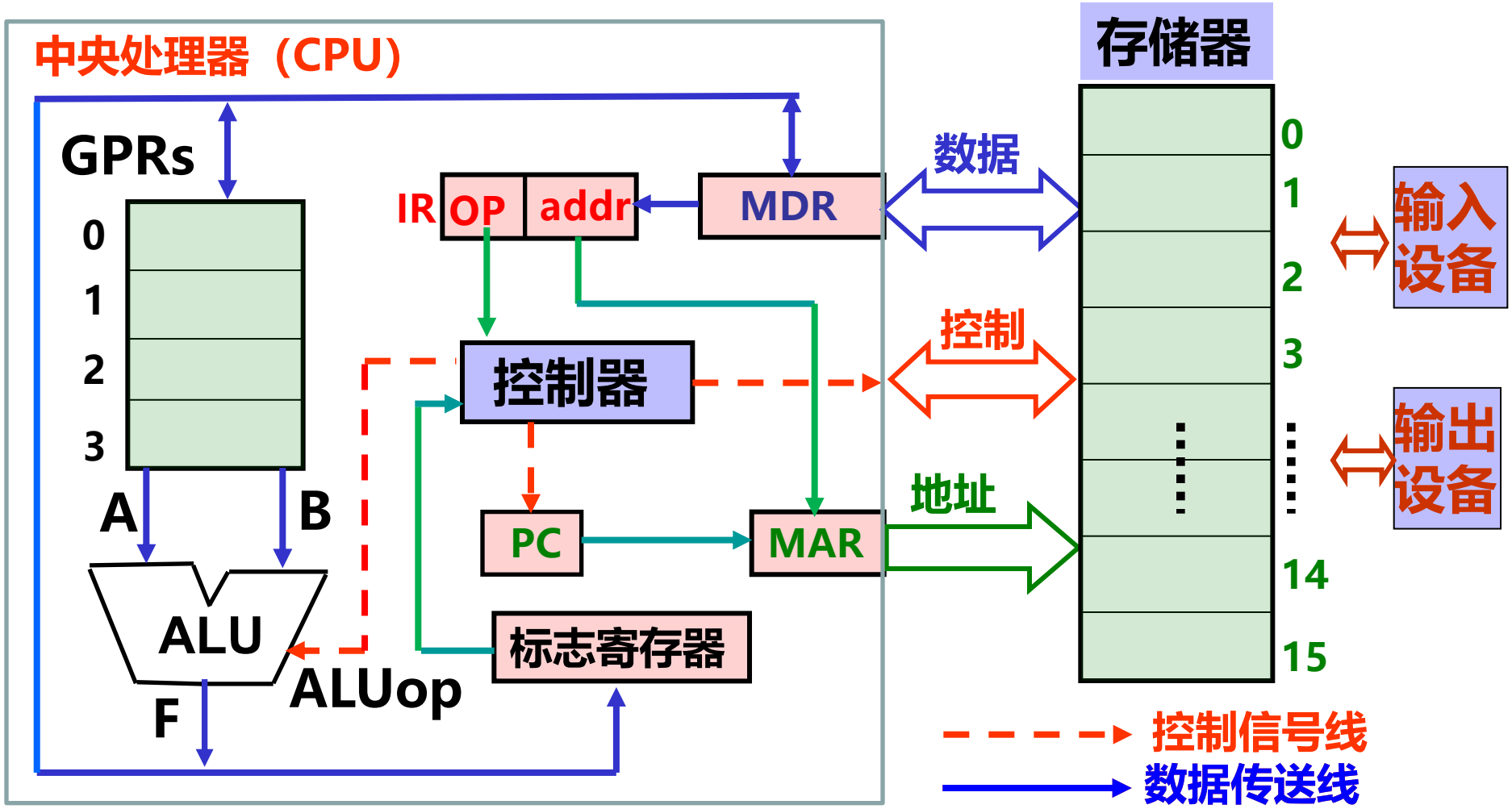

中央处理器（CPU）
GPRs
0
1
2
3
A
B
ALU
F
ALUop
IR
OP
addr
MDR
控制器
PC
MAR
标志寄存器
数据
控制
地址
存储器
0
1
2
3
14
15
输入设备
输出设备
控制信号线
数据传送线

认识计算机中最基本的部件

CPU：中央处理器；PC：程序计数器；MAR：存储器地址寄存器

ALU：算术逻辑部件；IR：指令寄存器；MDR：存储器数据寄存器

GPRs：通用寄存器组（由若干通用寄存器组成，早期就是累加器）



程序执行的基本过程

在加载程序时，设置需要执行的第一条指令的地址

地址存放在 程序计数器（PC）（即 x86中 EIP / RIP ）

- 根据 PC 取指令
- 对指令译码， PC ← PC + “1”
- 取操作数并执行

【对于条件转移指令，条件成立时，执行语句就是再次更新PC；条件不成立，则不会更新 PC】

【对于无条件转移指令、子程序调用和返回指令，执行语句会再次更新 PC 】

- 传送结果
- 重复上述过程，直到程序结束指令

以具体的例子讲解程序的执行过程

方法：程序的运行（反汇编）调试演示、讲解

- 掌握反汇编窗口、寄存器窗口、监视窗口、内存窗口等等的用法

```
#include <stdio.h>
int main(int argc, char* argv[])
{
        int  x, y, z;
        x = 10;
        y = 20;
        z = 3 * x + 6 * y + 4 * 8;
        printf("3*%d+6*%d+4*8=%d\n", x, y, z);
        return 0;
}
```

程序的运行

介绍内容：

- 汇编与反汇编
- 数据的地址及数据的表示形式

  局部变量的地址表达形式

  变量地址在监视窗口的显示

  局部变量的地址计算（EBP +n ）（验证地址表示形式）

  变量内容在内存中的显示

  监视窗口变量内容的显示（十进制，十六进制等）

汇编语言

将 某单元（[ebp-8]） 中的内容 置为 10。
C7 45 F8 0A 00 00 00

mov dword ptr [ebp-8],0Ah

int x=10; 变量 x 对应存储单元 [ebp-8]

mov x, 10 汇编语言源程序语句

汇编语言：用记忆符号书写的、主要操作与机器指令基本一一对应的、循一定语法规则的语言。

符号： 指令助记符、符号地址、标号等

汇 编：由汇编语言源程序 转换为 机器语言程序。

反汇编：由机器指令 转换为 汇编语言语句。

计算机工作基本原理

```
    int  x, y, z;
    x = 10;
00651865 C7 45 F8 0A 00 00 00 mov         dword ptr [ebp-8],0Ah
    y = 20;
0065186C C7 45 EC 14 00 00 00 mov         dword ptr [ebp-14h],14h
    z = 3 * x + 6 * y +
00651873 6B 45 F8 03
00651877 6B 4D EC 06                                          h],6
0065187B 8D 54 08 20
0065187F 89 55 E0                                             dx
    printf("3*%d+6*%d+4
```

寄存器

```
EAX = 0065C003 EBX = 00B6C000
ECX = 0065C003 EDX = 00000001
ESI = 00651023 EDI = 00D6FDB4
EIP = 00651865 ESP = 00D6FCC4
EBP = 00D6FDB4 EFL = 00000246
```

C01_走进计算机世界.exe | C01_走进计算机世界.cpp

```
00000c50  09 00 00 00 B8 CC CC CC  CC F3 AB B9 03 C0 41 00  ......
00000c60  E8 B1 FA FF FF C7 45 F8  0A 00 00 00 C7 45 EC 14  ......
00000c70  00 00 00 6B 45 F8 03 6B  4D EC 06 8D 54 08 20 89  ...kE.
00000c80  55 E0 8B 45 E0 50 8B 4D  EC 51 8B 55 F8 52 68 30  U..E.P
00000c90  7B 41 00 E8 35 F8 FF FF  83 C4 10 33 C0 5F 5E 5B  {A..5.
00000ca0  81 C4 E4 00 00 00 3B EC  E8 92 F9 FF FF 8B E5 5D  ......
00000cb0  C3 CC CC CC CC CC CC CC  CC CC CC CC CC CC CC CC  ......
00000cc0  CC CC CC CC CC CC CC CC  CC CC CC CC CC CC CC CC  ......
00000cd0  55 8B EC 81 EC E4 00 00  00 53 56 57 8D 7D DC B9  U....
```

程序的运行

在内存窗口观察，指令流

```
    int  x, y, z;
    x = 10;
00651865 C7 45 F8 0A 00 00 00 mov
    y = 20;
0065186C C7 45 EC 14 00 00 00 mov
    z = 3 * x + 6 * y + 4 * 8;
00651873 6B 45 F8 03          imul
00651877 6B 4D EC 06          imul
0065187B 8D 54 08 20          lea
0065187F 89 55 E0             mov
    printf("3*%d+6*%d+4*8=%d\n", x, y,
```

内存 1

地址: 0x00651865

0x00651865	c7	45	f8	0a	00	00	00
0x0065186C	c7	45	ec	14	00	00	00
0x00651873	6b	45	f8	03	6b	4d	ec
0x0065187A	06	8d	54	08	20	89	55
0x00651881	e0	8b	45	e0	50	8b	4d

分析指令机器码的组成部分：帮助理解指令译码

c7 45 f8 0a 00 00 00 c7 45 ec 14 00 00 00 6b ……

PC = 0x00651865

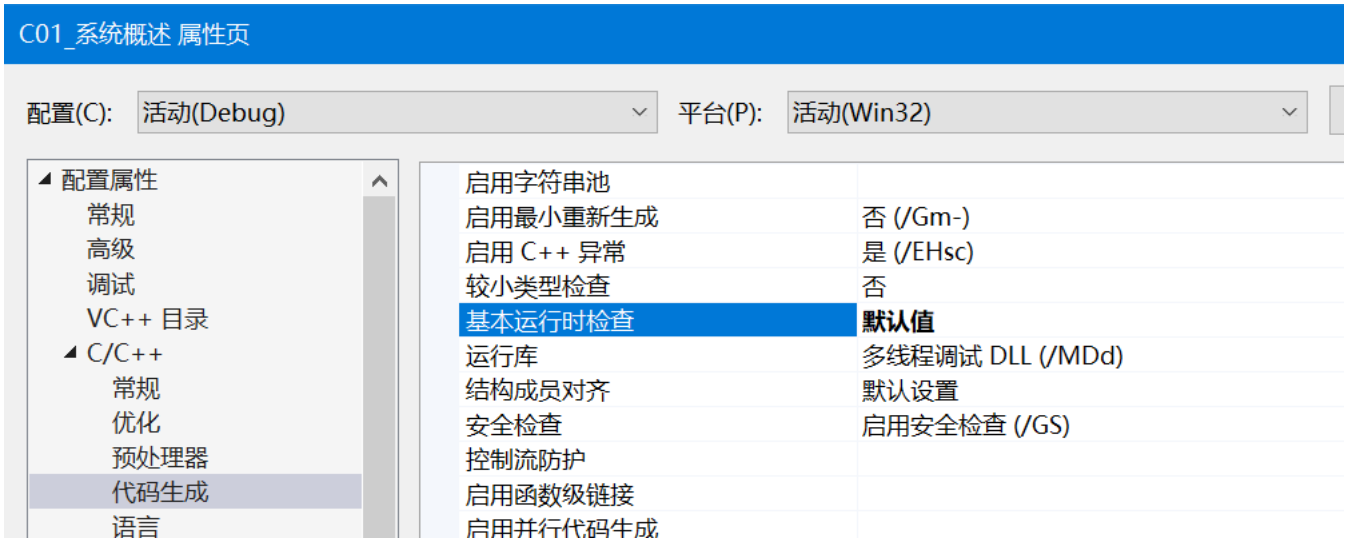
PC = 0x0065186C 译码后

指令译码，就相当于断句，并确定句子的主谓宾定状补。

程序的运行

分析变量之间的地址关系

- 比较不同编译开关下，编译器为变量安排空间的差异



- 比较x86, x64下，编译器为变量安排空间的差异

  理解地址分配的方案是多种多样的

程序的运行

- 在内存窗口，观察 printf 中格式串的存放
  了解参数传递的方法
- C语句与机器指令的对应关系
- 反汇编窗口，不同选项的设置，带来的窗口内容的变化
- 理解符号地址的概念

总结

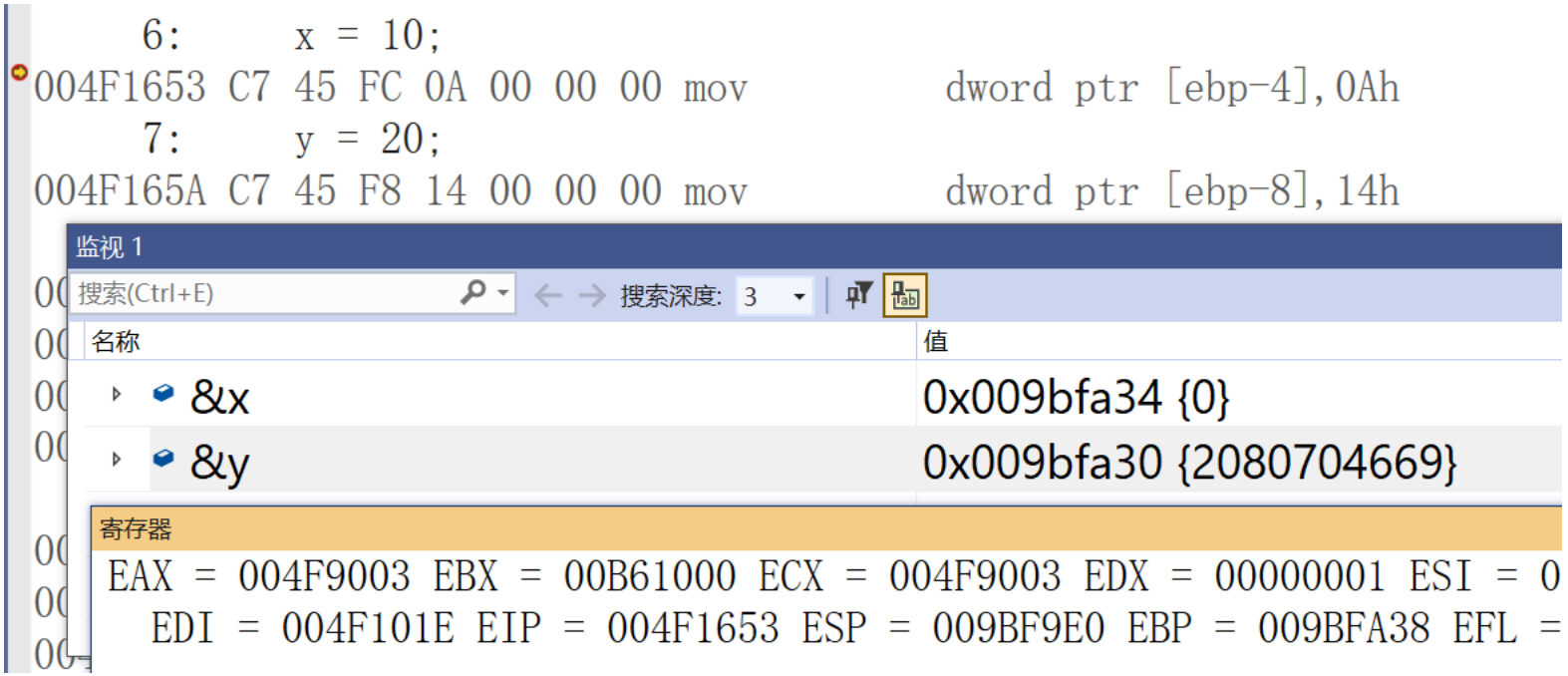
- 计算机是 0-1 世界，正确解读 字节流 的核心原则

  —— 上下文 + 编码规则

- 存储程序的基本工作原理 —— 编好程序+存储+自动执行
- 程序和指令的执行过程 ——程序计数器的更新方法和时机
- 正确理解地址的符号表示

  （寄存器也是符号；编程时定义的变量、标号都是符号）

测验



给出 ebp 的值，以及 x 的地址表达 [ebp - 4]，能计算出当前 x 的地址值吗？

给出 x 的地址表达 [ebp - 4]，当前 x 的地址值，能计算 ebp 中的值吗？

给出 x 的地址表达 [ebp - 4]，当前 x 的地址值，以及y 的地址表达 [ebp-8], 能计算出 y 的当前地址值吗？